문제 1) 재귀를 이용한 팩토리얼 계산

문제 설명

양의 정수 n을 입력받아 재귀 함수(Recursion)를 이용하여 n!을 계산하는 프로그램을 작성하시오.

팩토리얼은 다음과 같이 정의된다.

- 0! = 1
- n! = n × (n-1)!

실행 예시

정수 입력: 5

5! = 120

기본 코드

```
import java.util.Scanner;

public class Factorial {

    public static int factorial(int n) {

        // 코드 작성

    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("정수 입력: ");

        int n = sc.nextInt();

        System.out.println(n + "! = " + factorial(n));

    }

}
```

문제 2) 재귀를 이용한 피보나치 수열

문제 설명

정수 n을 입력받아 재귀 함수를 이용하여 n번째 피보나치 수를 구하는 프로그램을 작성하시오.

피보나치 수열은 다음과 같이 정의한다.

- F(0) = 0
- F(1) = 1
- F(n) = F(n-1) + F(n-2)

실행 예시

n 입력: 7

F(7) = 13

기본 코드

```
import java.util.Scanner;

public class Fibonacci {

    public static int fibonacci(int n) {

        // 코드 작성

    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("n 입력: ");

        int n = sc.nextInt();

        System.out.println("F(" + n + ") = " + fibonacci(n));

    }

}
```

문제 3) 배열을 이용한 리스트 구현

문제 설명

배열을 이용하여 문자열을 저장하는 리스트를 구현하시오.

- insert(String s, int i) 리스트의 i번째 위치에 문자열 s를 삽입.
- delete(int i) 리스트의 i번째 원소를 삭제.
- search(String s) 문자열 s가 저장되어 있는 위치를 반환, 존재하지 않는 경우 -1을 반환
- print() 리스트에 저장된 모든 문자열을 순서대로 출력.

기본 코드

```
class ArrayList {
    private String[] data;
    private int size;
    public ArrayList() {
        // 코드 작성
    }
    public void insert(String s, int i) {
        // 코드 작성
    }
    public void delete(int i) {
        // 코드 작성
    }
    public int search(String s) {
        // 코드 작성
    }
    public void print() {
        // 코드 작성
```

```
    }
}
public class Main {
    public static void main(String[] args) {
        ArrayList list = new ArrayList();
        list.insert("A", 0);
        list.insert("B", 1);
        list.insert("C", 2);
        System.out.print("초기 리스트: ");
        list.print();

        int i = list.search("C");
        System.out.println("C의 위치: " + i);

        list.insert("D", i);
        System.out.print("D 삽입 후: ");
        list.print();

        i = list.search("C");
        list.delete(i);
        System.out.print("C 삭제 후: ");
        list.print();
        }
}
```

문제 4) 연결 표현을 이용한 리스트 구현

문제 설명

단순 연결 리스트(Singly Linked List)를 이용하여 문자열을 저장하는 리스트를 구현하시오.

- addFirst(String s) 리스트의 첫 번째 노드로 문자열 s를 삽입.
- insert(String s, Node p) 노드 p의 다음 위치에 문자열 s를 저장한 새로운 노드를 삽입.
- delete(Node p) 노드 p의 다음 노드를 삭제.
- search(String s) 문자열 s가 저장되어 있는 Node를 반환, 존재하지 않는 경우 null을 반환.
- print() 리스트에 저장된 모든 문자열을 순서대로 출력.

기본 코드

```
class Node {
    String data;
    Node next;
    public Node(String data) {
        // 코드 작성
    }
}
class LinkedList {
    private Node head;
    private int size;
    public LinkedList() {
        // 코드 작성
    }
    public void addFirst(String s) {
        // 코드 작성
    }
```

```
    public void insert(String s, Node p) {
        // 코드 작성
    }
    public void delete(Node p) {
        // 코드 작성
    }
    public Node search(String s) {
        // 코드 작성
    }
    public void print() {    // 코드 작성
    }
}
public class Main {
    public static void main(String[] args) {
        LinkedList list = new LinkedList();
        list.addFirst("C");
        list.addFirst("B");
        list.addFirst("A");
        list.print();
        Node n = list.search("B");
        list.insert("D", n);
        list.print();
        list.delete(n);
        list.print();
    }
}
```